Loan & Credit Card Approval Prediction System Documentation

The Loan & Credit Card Approval Prediction System is a full-stack interactive workspace developed with React (Vite, TypeScript) on the frontend and an Express (Node.js) API backend. It integrates the Google Gemini Pro (via @google/genai) model to perform smart explanatory inference on credit risk decisions.
The application replicates the full machine learning lifecycle—ranging from database modeling (ER Diagrams), Exploratory Data Analysis (EDA), pipeline preprocessing, and machine learning model simulation, to live prediction scoring.

System Architecture & Workflow (The 5 Epics)

The application is structured around a 5-Epic Machine Learning Pipeline, accessible through the interactive header stepper on the dashboard:

`[Epic 1: ER Diagram] ➔ [Epic 2: Visualization] ➔ [Epic 3: Preprocessing] ➔ [Epic 4: Model Sandbox] ➔ [Epic 5: Live Prediction]`

Detailed Epic Breakdowns

Epic 1: Entity-Relationship Database (ER Diagram)

Models the backend storage layers required to support credit scoring.

- **Interactive ERD Board:** Visualizes the relational connections, primary keys (PK), foreign keys (FK), and cardinality mappings.
- **Schema Detail Inspector:** Explains columns, constraints, and relational properties for each table in a dedicated metadata console.

Epic 2: Data Visualizations & Descriptive Analysis (EDA)

Simulates Exploratory Data Analysis (EDA) conducted via Pandas, Matplotlib, and Seaborn:

- **Univariate Distributions:** Inspects single variable frequency profiles (e.g., OCCUPATION_TYPE and NAME_EDUCATION_TYPE).
- **Multivariate Correlation Matrix:** An interactive matrix mapping negative and positive associations between factors (e.g., Age vs. Employment Days, with a strong positive

+0.61 correlation).
- **Descriptive Summary Table:** Generates statistical metrics equivalent to a Pandas df.describe() query (Mean, Std Dev, Q1, Median, Q3, Max).

Epic 3: Preprocessing & Cleaning Board

An interactive step-by-step pipeline demonstrating data transformation techniques:

1. **Deduplication:** Drops duplicate records using a subset key.
2. **Null Imputation:** Flags or filters rows containing missing attributes.
3. **Day Absolute Normalization:** Converts negative days offsets (such as DAYS_BIRTH or DAYS_EMPLOYED) into absolute positive integers.
4. **Repayment Binarization:** Maps complex monthly credit history statuses into a binary classification variable: Approved (0) vs. Reject (1).
5. **Label Encoding:** Transforms text values into integer encodings suitable for machine learning algorithms.

Epic 4: Machine Learning Model Sandbox

A virtual model laboratory where developers can train and evaluate classic classification algorithms:

- **Algorithms Available:** Logistic Regression, Decision Tree, and Random Forest.
- **Interactive Hyperparameters:** Adjust regularizations (C-values), tree depths, and estimator ensemble sizes.
- **Performance Console:** Displays live validation metrics (Accuracy, Precision, Recall, F1 Score) alongside a holdout Confusion Matrix showing True/False Positives and Negatives.
- **Python Blueprint:** Generates ready-to-use Scikit-learn training code blocks tailored to your sandbox parameters.

Epic 5: Live Scoring Inference (Flask/Express Engine)

Simulates a live API-based production environment:

- **Interactive Scoring Form:** Captures real-time demographic variables (Income, Job tenure, Age, Housing, Marital status, Dependents, and repayment history).
- **AI-Explanatory Report:** Sends the parameter payload to the Express backend where the Gemini model evaluates credit risks and generates a professional, human-readable scoring decision summary.

Database Schema Blueprint (ERD Properties)

Entity Name	Primary Key (PK)	Foreign Keys (FK)	Relational Description
Users	UserID	None	Manages administrative credentials for officers and bank staff.
Applicant_Details	ApplicantID	UserID	Stores the demographic, educational, and household attributes of applicants (1-to-Many with Users).
Credit_History	HistoryID	ApplicantID	Relates monthly ledger payments of applicants (1-to-Many with Applicant_Details).
ML_Model	ModelID	None	Holds saved parameters, metadata, and testing accuracies of compiled algorithms.
Approval_Prediction	PredictionID	ApplicantID, ModelID	Connects models and applicants to store decision outcomes, scores, and explanations (1-to-1 with Applicants).

Project Directory Map

```
├── .env.example          # Sample configuration for credentials and environment paths
├── index.html            # Frontend DOM mount target
├── package.json          # Node project configuration, build scripts, and dependencies
├── server.ts             # Full-stack Express.js gateway server (Proxies prediction engine to
Gemini)
├── tsconfig.json         # TypeScript configuration
├── vite.config.ts        # Vite asset packager configuration
└── src/
    ├── main.tsx          # Main application entry point
    ├── index.css         # Global Tailwind CSS configurations
    ├── types.ts          # Core TypeScript model schemas and type declarations
    ├── App.tsx           # Main shell housing headers, workflow steppers, and layouts
    └── components/
        ├── ErDiagram.tsx         # Interactive ER schema board (Epic 1)
        ├── DataVisualization.tsx  # Univariate/multivariate graph engine (Epic 2)
        ├── DataPreprocessing.tsx  # Dynamic cleaning simulation layout (Epic 3)
        ├── ModelSandbox.tsx       # Hyperparameter sandbox & metric compiler (Epic 4)
        └── PredictionEngine.tsx   # Scoring form & explaining Gemini AI client (Epic 5)
```

Local Setup & Execution Guide (VS Code)

This section explains how to run the project locally on your machine and troubleshoot the specific node execution errors commonly found in Windows PowerShell environments.

1. Prerequisites

Ensure you have Node.js (v18 or higher recommended) and npm installed:

```
node -v
npm -v
```

2. Configure Environment Secrets

Create a .env file in the project's root directory and populate it with your credentials:

```
GEMINI_API_KEY="YOUR_ACTUAL_GEMINI_API_KEY"
```

```
APP_URL="http://localhost:3000"
```

3. Install Dependencies

Open your terminal in VS Code inside the project directory and run:

```
npm install
```

Troubleshooting: Solving the Windows tsx Error

Why the error occurs: When running npm run dev on Windows, you encountered the following error:

```
'-credit-card-approval-prediction-system\node_modules\.bin\' is not recognized as an internal or external command...
Cannot find module 'D:\suvignesh\tsx\dist\cli.mjs'
```

This is a common issue on Windows machines caused by special characters (like &) in directory names, space/path parsing errors in Windows shells, or missing local dev dependency paths.

Solution Options

Option A: Rename Your Project Folder (Highly Recommended)
Windows command prompts and PowerShell often fail to resolve relative binary paths inside folders containing special characters like &.

1. Close VS Code.
2. Rename the project folder from: D:\suvignesh\loan-&-credit-card-approval-prediction-system to a clean, alphanumeric name: D:\suvignesh\loan-credit-prediction-system
3. Re-open VS Code in the renamed folder and run npm run dev.

Option B: Run via Direct Node Execution (Bypasses tsx path issues)
If you cannot rename the directory, you can execute the server using Node directly by telling it to parse TypeScript files. Run this command in your VS Code terminal:

```
npx tsx server.ts
```

Or use the global npx runner to bypass pathing issues:

```
npx --yes tsx server.ts
```

Option C: Update your package.json Scripts for Windows Compatibility
You can change the "dev" script in package.json to leverage node directly, which avoids shell

resolution issues. Open package.json and change the "dev" line:
From:

```
"dev": "tsx server.ts"
```

To:

```
"dev": "node --import tsx server.ts"
```

Or compile and run the production server output:

```
npm run build
npm run start
```

Summary of Final Deliverables

- **Relational Database ERD Interface:** Provides a visual baseline explaining the table architecture of financial scoring schemas.
- **Interactive EDA Suite:** Built-in distributions, dynamic correlation maps, and Pandas-style statistical summaries.
- **Interactive Data Cleaning System:** Guides users through standard Scikit-learn data scrubbing pipelines.
- **Interactive Hyperparameter Sandbox:** Simulates classifier compiles for Logistic Regression, Decision Trees, and Random Forests.
- **AI Inference Gateway:** Combines Express APIs with Gemini AI to deliver real-time risk decision explanations.